

Module 12 - chapter 9

Variables in Terraform

Variables -> input variables

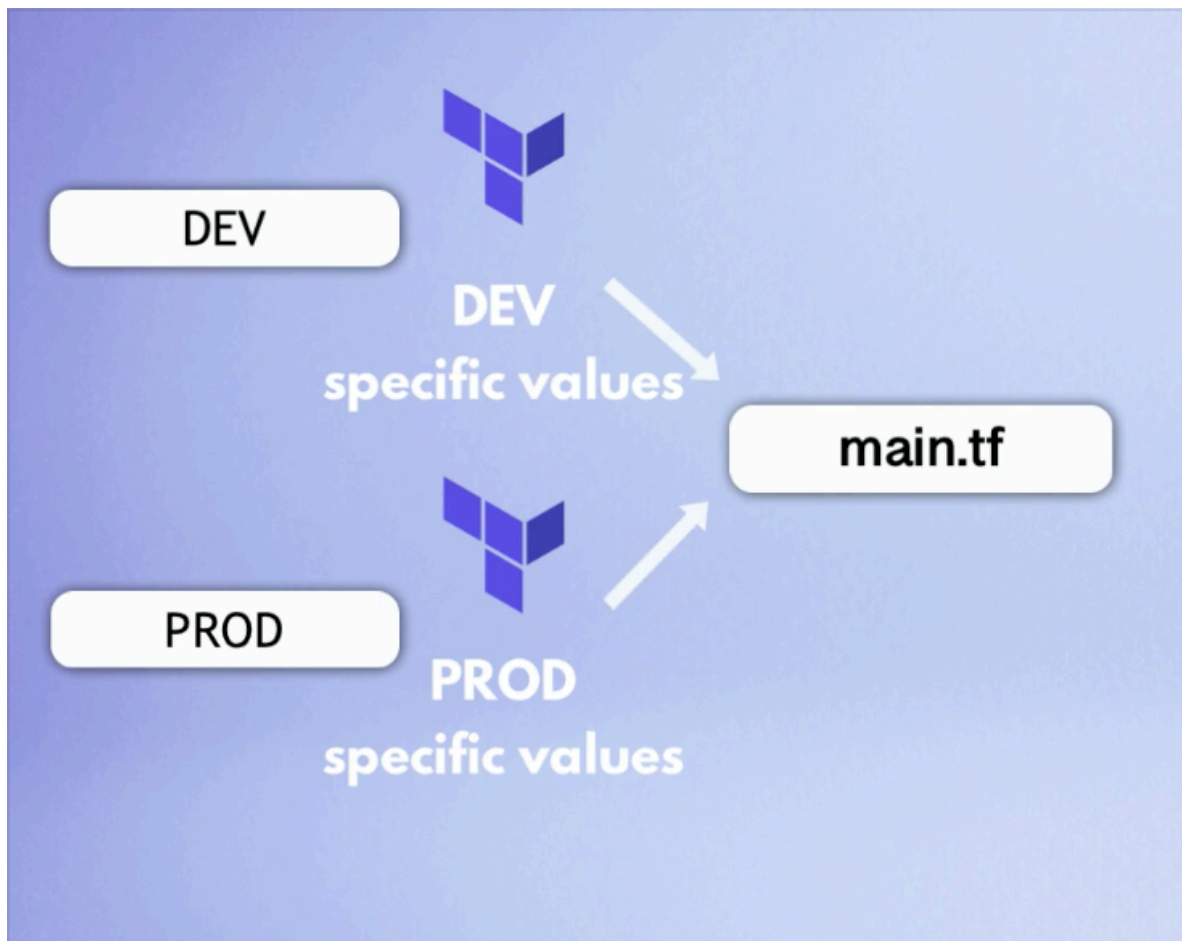
For example we want to pass the CIDR range as parameter instead of hardcoded.

Input variables are like function arguments.

Good for:

- Reusability

We can use the same template for different environments: DEV, QA and PROD. And we can pass arguments to each depending on the environment:



Let's see it with an example -> cidr_block attribute will get an input variable

First we define the variable before any resource

```

1  provider "aws" {
2      region = "eu-west-2"
3  }
4
5  variable "subnet_cidr_block" {
6      description = "subnet cidr block"
7  }
8

```

Then we replace the hardcoded value for the new variable using the keyword 'var' and the name of the variable

```

16  resource "aws_subnet" "dev-subnet-1" {
17      vpc_id = aws_vpc.development-vpc.id
18      cidr_block = var.subnet_cidr_block
19      availability_zone = "eu-west-2a"
20      tags = {
21          Name: "subnet-1-dev"
22      }
23  }
24

```

Next we assign a value to the variable

There are 3 ways

3 ways to pass value to the input variable

1. When executing -> terraform apply

We get a prompt asking to enter the value for the variable

```

→ terraform git:(Module_12/Terraform_chapter-9) ✕ terraform apply
var.subnet_cidr_block
  subnet cidr block

Enter a value:

```

And I pass a value:

```

→ terraform git:(Module_12/Terraform_chapter-9) ✕ terraform apply
var.subnet_cidr_block
  subnet cidr block

Enter a value: 10.0.20.0/24

```

And now we see a new symbol '-/+' for replacement -> first destroy then create

```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# aws_subnet.dev-subnet-1 must be replaced
-/+ resource "aws_subnet" "dev-subnet-1" {
  ~ arn                                = "arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-027536d7a88db25b0" -> (known a
fter apply)
  ~ availability_zone_id              = "euw2-az2" -> (known after apply)
  ~ cidr_block                       = "10.0.10.0/24" -> "10.0.20.0/24" # forces replacement
  ~ enable_lni_at_device_index       = 0 -> null
  ~ id                               = "subnet-027536d7a88db25b0" -> (known after apply)
  + ipv6_cidr_block                   = (known after apply)
  + ipv6_cidr_block_association_id    = (known after apply)
  ~ map_customer_owned_ip_on_launch  = false -> null
  ~ owner_id                         = "706976333940" -> (known after apply)
  ~ private_dns_hostname_type_on_launch = "ip-name" -> (known after apply)
  ~ tags                             = {
    "Name" = "subnet-1-dev"
  }
}
# (12 unchanged attributes hidden)

```

So 'dev-subnet- 1' with hardcoded cidr_block is destroyed and the resource as created again using the input variable.

```
Plan: 1 to add, 0 to change, 1 to destroy.
```

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

```
aws_subnet.dev-subnet-1: Destroying... [id=subnet-027536d7a88db25b0]
```

```
aws_subnet.dev-subnet-1: Destruction complete after 1s
```

```
aws_subnet.dev-subnet-1: Creating...
```

```
aws_subnet.dev-subnet-1: Creation complete after 0s [id=subnet-031e68203403abb0d]
```

```
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

And if we check AWS UI we can see the new subnet with the new CIDR block value (and also the id is the same as the one from the terminal)

The screenshot shows the AWS Management Console interface for Subnets. At the top, there's a search bar and a table of subnets. The table has columns for Name, Subnet ID, State, VPC, Block Public..., IPv4 CIDR, IPv6 CIDR, and IPv6. The first row is selected, showing 'subnet-1-dev' with Subnet ID 'subnet-031e68203403abb0d', State 'Available', VPC 'vpc-0aeaab4662d170ab3 | development', Block Public... 'Off', IPv4 CIDR '10.0.20.0/24', and IPv6 CIDR '-'. Below the table, the details for 'subnet-031e68203403abb0d / subnet-1-dev' are shown. The details section includes tabs for Details, Flow logs, Route table, Network ACL, CIDR reservations, Sharing, and Tags. The Details tab is active, showing information like Subnet ID, Subnet ARN, State (Available), IPv4 CIDR (10.0.20.0/24), Availability Zone (euw2-az2 (eu-west-2)), Subnet ARN (arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-031e68203403abb0d), Available IPv4 addresses (251), Network border group (eu-west-2), State (Available), IPv6 CIDR (-), VPC (vpc-0aeaab4662d170ab3 | development), Block Public Access (Off), IPv6 CIDR association ID (-), and Route table (rtb-033a03f4197c46804).

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR	IPv6
subnet-1-dev	subnet-031e68203403abb0d	Available	vpc-0aeaab4662d170ab3 development	Off	10.0.20.0/24	-	-
-	subnet-03da2ce00b3dba523	Available	vpc-03677c88f7cc2f715	Off	172.31.0.0/20	-	-
-	subnet-092c85ea013cbc240	Available	vpc-03677c88f7cc2f715	Off	172.31.16.0/20	-	-
-	subnet-051c96240932f9c7e	Available	vpc-03677c88f7cc2f715	Off	172.31.32.0/20	-	-

subnet-031e68203403abb0d / subnet-1-dev

Details | Flow logs | Route table | Network ACL | CIDR reservations | Sharing | Tags

Details

Subnet ID subnet-031e68203403abb0d	Subnet ARN arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-031e68203403abb0d	State Available	Block Public Access Off
IPv4 CIDR 10.0.20.0/24	Available IPv4 addresses 251	IPv6 CIDR -	IPv6 CIDR association ID -
Availability Zone euw2-az2 (eu-west-2)	Network border group eu-west-2	VPC vpc-0aeaab4662d170ab3 development	Route table rtb-033a03f4197c46804

This way is good for testing but is not efficient.

If we want to automate our processes we need to define our variables without using a user prompt.

2. Command line argument -> terraform apply -var "<name of var>=value"

-> terraform apply -var "subnet_cidr_block=10.0.30.0/24"

```
main.tf x
1 provider "aws" {
2     region = "eu-west-2"
3 }
4
5 variable "subnet_cidr_block" { You, 9 minutes ago • Uncommitted ch
6     description = "subnet cidr block"
7 }
8
9 resource "aws_vpc" "development-vpc" {
10     cidr_block = "10.0.0.0/16"
11     tags = {
12         Name: "development"
13     }
14 }
15
16 variable subnet_cidr_block

after 0s [id=subnet-031e68203403abb0d]

ged, 1 destroyed.
ter-9) x clear
ter-9) x terraform apply -var "subnet_cidr_block=10.0.30.0/24"
```

And TF says it will replace:

```
+ terraform git:(Module_12/Terraform_chapter-9) x terraform apply -var "subnet_cidr_block=10.0.30.0/24"
aws_vpc.development-vpc: Refreshing state... [id=vpc-0aeaab4662d170ab3]
aws_subnet.dev-subnet-1: Refreshing state... [id=subnet-031e68203403abb0d]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# aws_subnet.dev-subnet-1 must be replaced
-/+ resource "aws_subnet" "dev-subnet-1" {
  ~ arn                                = "arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-031e68203403abb0d" -> (known a
fter apply)
  ~ availability_zone_id              = "euw2-az2" -> (known after apply)
  ~ cidr_block                        = "10.0.20.0/24" -> "10.0.30.0/24" # forces replacement
  ~ enable_lni_at_device_index       = 0 -> null
  ~ id                               = "subnet-031e68203403abb0d" -> (known after apply)
  + ipv6_cidr_block                  = (known after apply)
  + ipv6_cidr_block_association_id   = (known after apply)
  - map_customer_owned_ip_on_launch  = false -> null
```

And it is done

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

aws_subnet.dev-subnet-1: Destroying... [id=subnet-031e68203403abb0d]

aws_subnet.dev-subnet-1: Destruction complete after 1s

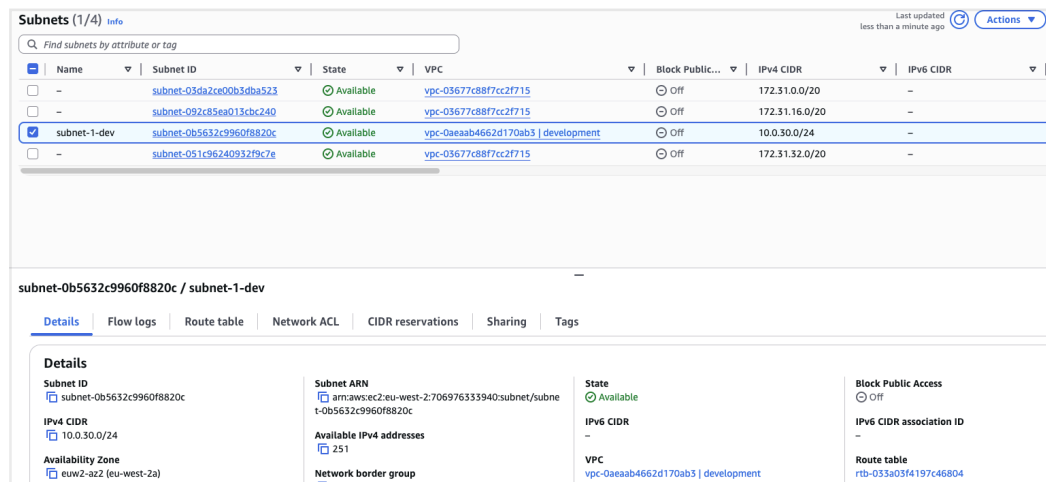
aws_subnet.dev-subnet-1: Creating...

aws_subnet.dev-subnet-1: Creation complete after 0s [id=subnet-0b5632c9960f8820c]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

→ terraform git:(Module_12/Terraform_chapter-9) ×

And we check again AWS UI and it is done

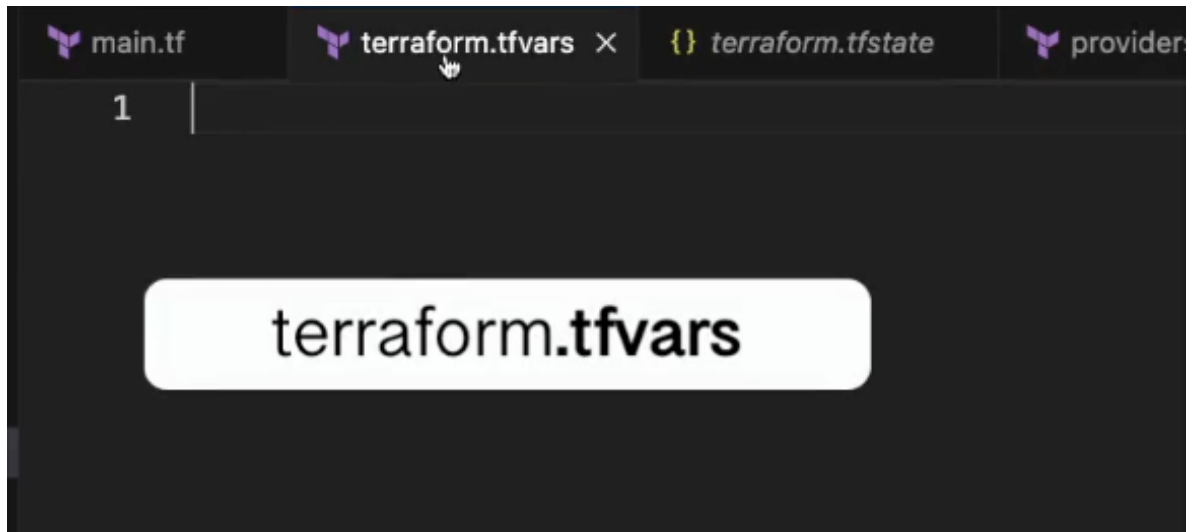


Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR
-	subnet-03da2ce00b3dba523	Available	vpc-03677c88f7cc2f715	Off	172.31.0.0/20	-
-	subnet-092c85ea013cbc240	Available	vpc-03677c88f7cc2f715	Off	172.31.16.0/20	-
subnet-1-dev	subnet-0b5632c9960f8820c	Available	vpc-0aeaaab4662d170ab3 development	Off	10.0.30.0/24	-
-	subnet-051c96240932f9c7e	Available	vpc-03677c88f7cc2f715	Off	172.31.32.0/20	-

subnet-0b5632c9960f8820c / subnet-1-dev			
Details			
Subnet ID subnet-0b5632c9960f8820c	Subnet ARN arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-0b5632c9960f8820c	State Available	Block Public Access Off
IPv4 CIDR 10.0.30.0/24	Available IPv4 addresses 251	IPv6 CIDR -	IPv6 CIDR association ID -
Availability Zone euw2-az2 (eu-west-2a)	Network border group -	VPC vpc-0aeaaab4662d170ab3 development	Route table rtb-035a03f4197c46804

3. Define a variables file -> best way

The name of the file has a convention and it has to be: <name>.tfvars it must have the extension .tfvars



Q:

I wonder if the name can be anything? like dev.tfvars or it has to be terraform.tfvars

A:

- The name can be anything with .tfvars extension, but with one important distinction:

Automatically loaded (no flag needed):

- terraform.tfvars
- terraform.tfvars.json
- Any file ending in .auto.tfvars (e.g. dev.auto.tfvars)

Manually specified (requires -var-file flag):

- Any other name like dev.tfvars, prod.tfvars, staging.tfvars — you need to pass it explicitly:

```
terraform apply -var-file="dev.tfvars"
```

So a common pattern for managing multiple environments is:

dev.tfvars

staging.tfvars

prod.tfvars

And then run:

```
terraform apply -var-file="dev.tfvars"
```

This way you keep environment-specific values separate and choose which one to load at runtime.

So I created my vars file and add the key-value pair for my variables (I only have one)

```
main.tf x terraform.tfvars x
1 subnet_cidr_block = "10.0.40.0/24" Yo
```

-> terraform apply

And it is done

```
Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

aws_subnet.dev-subnet-1: Destroying... [id=subnet-0b5632c9960f8820c]
aws_subnet.dev-subnet-1: Destruction complete after 1s
aws_subnet.dev-subnet-1: Creating...
aws_subnet.dev-subnet-1: Creation complete after 1s [id=subnet-098d57e55f18bfa4c]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

And we check AWS UI

Subnets (1/4) info									
Find subnets by attribute or tag									
☐	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR		
☐	-	subnet-03da2ce00b3dba523	Available	vpc-03677c88f7cc2f715	Off	172.31.0.0/20	-		
☐	-	subnet-092c85ea013bc240	Available	vpc-03677c88f7cc2f715	Off	172.31.16.0/20	-		
☐	-	subnet-051c96240932f9c7e	Available	vpc-03677c88f7cc2f715	Off	172.31.32.0/20	-		
☒	subnet-1-dev	subnet-098d57e55f18bfa4c	Available	vpc-0aeab4662d170ab3 development	Off	10.0.40.0/24	-		

subnet-098d57e55f18bfa4c / subnet-1-dev

Details | Flow logs | Route table | Network ACL | CIDR reservations | Sharing | Tags

Details

Subnet ID

subnet-098d57e55f18bfa4c

Subnet ARN

arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-098d57e55f18bfa4c

State

Available

Block Public Access

Off

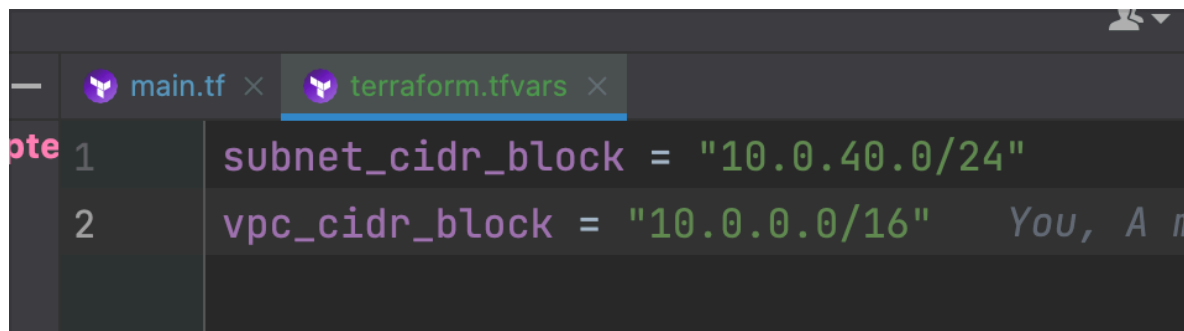
Now let's create a new variable for the cidr block of the vpc:


```

4
5   variable "subnet_cidr_block" {
6       description = "subnet cidr block"
7   }
8
9   variable "vpc_cidr_block" {
10      description = "vpc cidr block"
11  }
12
13  resource "aws_vpc" "development-vpc" {
14      cidr_block = var.vpc_cidr_block
15      tags = {
16          Name: "development"

```

And update the vars file



```

1  subnet_cidr_block = "10.0.40.0/24"
2  vpc_cidr_block = "10.0.0.0/16"

```

This way we can parameterised our TF file depending on each use case.

Use Case for Input Variables

One Use case is when we have a config file to create and manage an infra and then we want to replicate the same infrastructure for different environments.

Replicate same infrastructure for different environments

For example name of the VPC to have the env name:

```
12
13 variable "environment" {  You, Moments ago • Uncommitted
14     description = "deployment environment"
15 }
16
17 resource "aws_vpc" "development-vpc" {
18     cidr_block = var.vpc_cidr_block
19     tags = {
20         Name: var.environment
21     }
22 }
```

And we update the vars file

```
main.tf x terraform.tfvars x
1 subnet_cidr_block = "10.0.40.0/24"
2 vpc_cidr_block = "10.0.0.0/16"
3
4 environment = "development"  You, Moments ago •
```

And we will make the resource name more generic

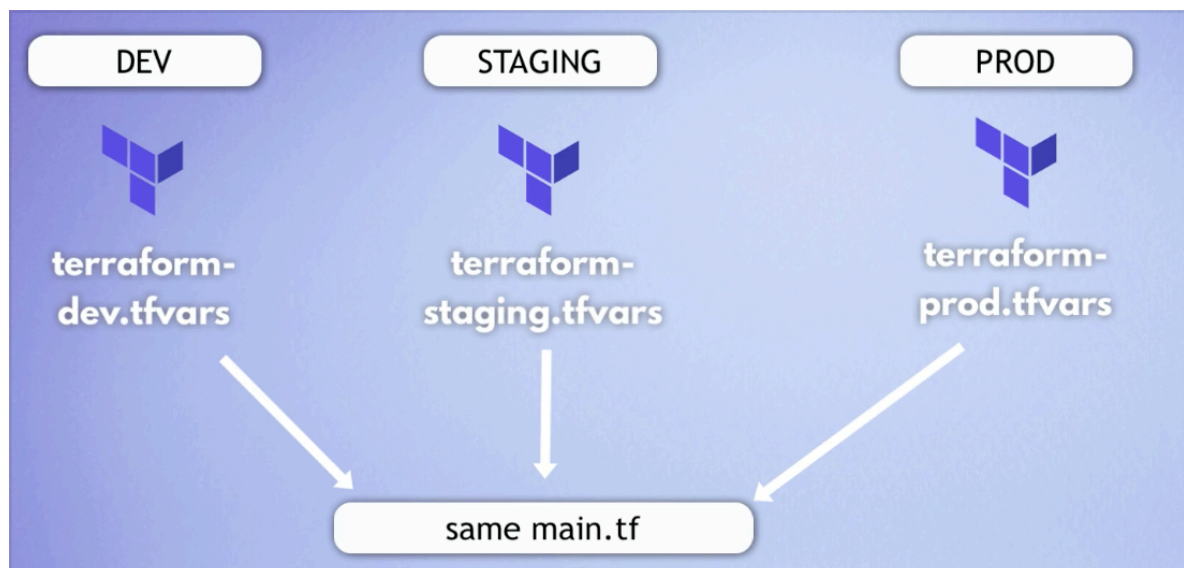
So Instead of "development-vpc" we rename it "myapp-vpc" for example so the config is reusable for multiple envs:

```
15 variable "environment" {
16   description = "deployment environment"
17 }
18
19 resource "aws_vpc" "development-vpc" {
20   cidr_block = var.vpc_cidr_block
21   tags = {
22     Name: var.environment
23   }
24 }
25
26 resource "aws_subnet" "dev-subnet-1" {
27   vpc_id = aws_vpc.development-vpc.id
28   cidr_block = var.subnet_cidr_block
29   availability_zone = "eu-central-1a"
30   tags = {
31     Name: "subnet-1-dev"
32   }
33 }
```

"myapp-vpc"

"myapp-subnet-1"

So we will have a variable file for each env

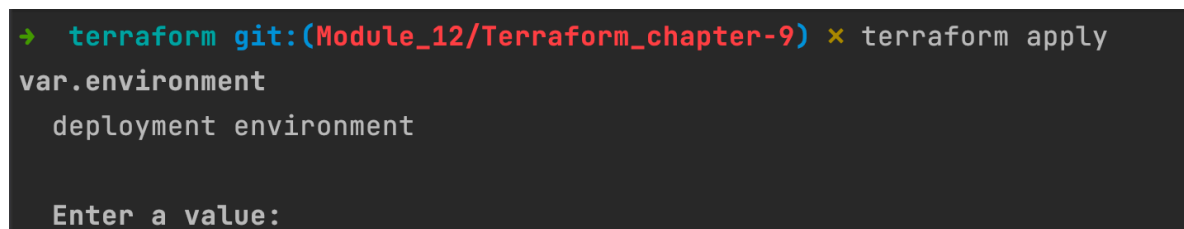


Terraform by default looks for terraform.tfvars and if the name of the file is not that like terraform-dev.tfvars



```
main.tf x terraform-dev.tfvars x
1 subnet_cidr_block = "10.0.40.0/24"
2 vpc_cidr_block = "10.0.0.0/16"
3
4 environment = "development" You, Mome
```

Then when we -> terraform apply then it will prompt us to give values to the vars



```
→ terraform git:(Module_12/Terraform_chapter-9) x terraform apply
var.environment
  deployment environment

Enter a value:
```

So when the var file has different naming to the default terraform.tfvars we need to pass the file name in the command line using the flag -var-file

-> terraform apply -var-file terraform-dev.tfvars



```
→ terraform git:(Module_12/Terraform_chapter-9) x terraform apply -var-file terraform-dev.tfvars
aws_vpc.development-vpc: Refreshing state... [id=vpc-0aeaab4662d170ab3]
aws_subnet.dev-subnet-1: Refreshing state... [id=subnet-098d57e55f18bfa4c]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
→ terraform git:(Module_12/Terraform_chapter-9) x
```

Default values

Variables can have a default value. So it will be used if TF can't find a value in the vars file and it won't prompt us either.

```
4
5  variable "subnet_cidr_block" {
6      description = "subnet cidr block"
7      default = "10.0.10.0/24"
8  }
9
```

A default value makes the variable -> Optional



A default value makes the variable **optional**

Type constraints

We can also set a -> variable type

The screenshot shows a Terraform configuration file with the following content:

```
1 provider "aws" {
2   region = "eu-central-1"
3   access_key = "AKIAUZ7NN5BBI..."
4   secret_key = "HCVKNGYvYzKU5..."
5 }
6
7 variable "subnet_cidr_block" {
8   description = "subnet cidr b..."
9   default = "10.0.10.0/24"
10  type =
11 }
12
13 resource "aws_vpc" "development" {
14   cidr_block = "10.0.0.0/16"
15   tags = {
16     Name = "development"
17   }
18 }
19
20 resource "aws_subnet" "dev-sub" {
21   vpc_id = aws_vpc.development
22   cidr_block = var.subnet_cidr_block
23   availability_zone = "eu-cent..."
24   tags = {
```

A pop-up window titled "Possible Types" is overlaid on the right side of the code editor. It contains the following text:

Possible Types

A *primitive* type is a simple type that isn't made from any other types. All primitive types in Terraform are represented by a type keyword. The available primitive types are:

- `string`: a sequence of Unicode characters representing some text, such as `"hello"`.
- `number`: a numeric value. The `number` type can represent both whole numbers like `15` and fractional values such as `6.283185`.
- `bool`: either `true` or `false`. `bool` values can be used in conditional logic.

Conversion of Primitive Types

The Terraform language will automatically convert `number` and `bool` values to `string` values when needed, and vice-versa as long as the string contains a valid representation of a number or boolean value.

- `true` converts to `"true"`, and vice-versa
- `false` converts to `"false"`, and vice-versa
- `15` converts to `"15"`, and vice-versa

For a cidr_block is has to be 'string'

```
4
5 variable "subnet_cidr_block" {
6   description = "subnet cidr block"
7   default = "10.0.10.0/24"
8   type = string
9 }
10
```

And by adding the type we enforced that the users pass a 'string' to that variable



"type" specifies
what value types are accepted

And TF will throw an error if pass an array for example

```
main.tf x terraform-dev.tfvars x
1 subnet_cidr_block = ["10.0.50.0/24"] You, 5
```

Let's try

-> terraform apply -var-file terraform-dev.tfvars

```
terraform git:(Module_12/Terraform_chapter-9) x terraform apply -var-file terraform-dev.tfvars
aws_vpc.development-vpc: Refreshing state... [id=vpc-0aeaab4662d170ab3]

Planning failed. Terraform encountered an error while generating this plan.

Error: Invalid value for input variable

on terraform-dev.tfvars line 1:
1: subnet_cidr_block = ["10.0.50.0/24"]

The given value is not suitable for var.subnet_cidr_block declared at main.tf:5,1-29: string required, but have tuple.

terraform git:(Module_12/Terraform_chapter-9) x
```

If we want to pass a list then we can do the following -> list(type)

```

7  variable "subnet_cidr_block" {
8    description = "subnet cidr block"
9    default = "10.0.10.0/24"
10   type = list(string)
11 }
12

```

list

resource-random-string Resource random string

So I make editions to the variable:

```

4
5  variable "cidr_blocks" {
6    description = "cidr blocks for vpc and subnets"
7    type = list(string)
8  }

```

And we rename the variable as we will use it for vpc and subnets

And I update the vars file

```

main.tf x terraform-dev.tfvars x
1  cidr_blocks = ["10.0.0.0/16", "10.0.10.0/24"]

```

Item [0] will be for the vpc and item[1] for on of its subntes

So now we can use the variable as reference with a twist as it is a list. We use a list syntax [item location]

```

17  resource "aws_subnet" "dev-subnet-1" {
18    vpc_id = aws_vpc.development-vpc.id
19    cidr_block = var.cidr_blocks[1]
20    availability_zone = "eu-west-2a"
21    tags = {
22      Name: "subnet-1-dev"
23    }
24  }

```

And for the vpc


```

10 resource "aws_vpc" "development-vpc" {
11     cidr_block = var.cidr_blocks[0]
12     tags = {
13         Name: "development"
14     }
15 }
16

```

-> terraform apply -var-file terraform-dev.tfvars

```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# aws_subnet.dev-subnet-1 must be replaced
-/+ resource "aws_subnet" "dev-subnet-1" {
    ~ arn                                = "arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-098d57e55f18bfa4c" -> (known after apply)
    ~ availability_zone_id               = "euw2-az2" -> (known after apply)
    ~ cidr_block                        = "10.0.40.0/24" -> "10.0.10.0/24" # forces replacement
    - enable_lni_at_device_index        = 0 -> null
    ~ id                                = "subnet-098d57e55f18bfa4c" -> (known after apply)
    + ipv6_cidr_block                    = (known after apply)
    + ipv6_cidr_block_association_id    = (known after apply)
}

```

We only changed the cidr block value for the subnet, so that is the only change

And it is done

```

Enter a value: yes

aws_subnet.dev-subnet-1: Destroying... [id=subnet-098d57e55f18bfa4c]
aws_subnet.dev-subnet-1: Destruction complete after 0s
aws_subnet.dev-subnet-1: Creating...
aws_subnet.dev-subnet-1: Creation complete after 1s [id=subnet-03f2aed3588c301fd]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

```

Let's check AWS UI

Subnets (1/4) info

Find subnets by attribute or tag

Last updated less than a minute ago

Actions Create

	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR	IPv6 C
☐	-	subnet-03da2ce00b3dba523	Available	vpc-03677c88f7cc2f715	Off	172.31.0.0/20	-	-
☐	-	subnet-092c85ea013cbc240	Available	vpc-03677c88f7cc2f715	Off	172.31.16.0/20	-	-
☒	subnet-1-dev	subnet-03f2aed3588c301fd	Available	vpc-0aeaab4662d170ab3 development	Off	10.0.10.0/24	-	-
☐	-	subnet-051c96240932f9c7e	Available	vpc-03677c88f7cc2f715	Off	172.31.32.0/20	-	-

subnet-03f2aed3588c301fd / subnet-1-dev

Details Flow logs Route table Network ACL CIDR reservations Sharing Tags

Details

Subnet ID subnet-03f2aed3588c301fd	Subnet ARN arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-03f2aed3588c301fd	State Available	Block Public Access Off
---------------------------------------	--	--------------------	----------------------------

We can also have OBJECTS as variables in TF

```

1 provider "aws" {
2   region = "eu-central-1"
3   access_key = "AKIAUZ7NN5BBID2IQDHY"
4   secret_key = "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"
5 }
6
7 variable "cidr_blocks" {
8   description = "cidr blocks for vpc and subnets"
9   type = list(string)
10 }
11

```

object

So we can create a list of objects

```

1 cidr_blocks = [
2   {cidr_block = "10.0.0.0/16", name = "dev-vpc"},
3   {cidr_block = "10.0.10.0/24", name = "dev-subnet"}
4 ]

```

list of objects

Now we update the usage of the variable in the main.tf

```

5 variable "cidr_blocks" {
6   description = "cidr blocks for vpc and subnets"
7   type = list(object)
8 }

```

You, 7 minutes ago • Uncommitted

And we can do better, we can validate the objects, we can make sure the attributes of the objects have a specific type for example

```

4
5     variable "cidr_blocks" {
6         description = "cidr blocks for vpc and subnets"
7         type = list(object({
8             vpc_cidr = string
9             name = string
10        }))
11    }
12

```

So we expect a list of objects with two attributes and both of string type.

And a final edit is to update how the value is passed to the resource:

```

12
13     resource "aws_vpc" "development-vpc" {
14         cidr_block = var.cidr_blocks[0].cidr_block
15         tags = {
16             Name: var.cidr_blocks[0].name
17         }
18     }
19
20     resource "aws_subnet" "dev-subnet-1" {
21         vpc_id = aws_vpc.development-vpc.id
22         cidr_block = var.cidr_blocks[1].cidr_block
23         availability_zone = "eu-west-2a"
24         tags = {
25             Name: var.cidr_blocks[1].name
26         }
27     }

```

So we have to add the attribute that we are passing from the object.

And we also updated the description of the variable:

```
4
5  variable "cidr_blocks" {
6      description = "cidr blocks and name tags for vpc and subnets"  Yo
7      type = list(object({
8          vpc_cidr = string
9          name = string
10     }))
11 }
```